

Contents

U01, Mathematical language	3
U02, Linear geometry and decompositions	4
U03, Calculus and optimization	5
U04, Probability, density, and estimation	6
U05, Risk, regularization, and generalization	7
U06, Linear models, kernels, and margins	8
U07, Trees and ensembles	9
U08, Neural and sequence architectures	10
U09, Unsupervised and latent models	11
U10, Generative bridge and mathematical synthesis	12
Where to go next	13
Cheatsheet: formulas, shapes, traps, decision rules	14
U01, mathematical language	15
U02, linear geometry	16
U03, calculus and optimization	17
U04, probability and estimation	18
U05, risk and generalization	19
U06, linear models, kernels, margins	20
U07, trees and ensembles	21
U08, neural and sequence	22
U09, unsupervised and latent	23
U10, generative synthesis	24
Cross-unit decision rules	25

Crash course: the whole course in thirty minutes

Mathematical Foundations of Machine Learning, Stanford Frontier AI

Baseline: October 6, 2026. Print companion to the HTML course site.

Ten units, three minutes each. Every worked number below comes from the named lesson section. The crash course points, the lessons prove.

U01, Mathematical language

The course speaks in sets, functions, sums, logs, and vectors. Learn the notation once: sum over an index, log base e unless stated, vectors as columns. Units attach to every number. A plot earns trust only when the axes, the data, and the code that drew it are all visible.

Worked number (U01-C03): 65 thousand dollars for 150 sq ft gives $65 / 150 = 0.4333$ thousand dollars per sq ft. Multiply back: $0.4333 * 150 = 65.0$. The round trip closes, so the unit arithmetic is consistent.

Memory aid. Units first. Sums explicit. Asserts everywhere.

Object	Notation in this course	Meaning
Set	$\{1, 2, 3\}$	A collection of distinct elements
Function	$f: X \rightarrow Y$	A rule that maps each input to one output
Sum	$\text{sum}_i x_i$	Add over the index i
Natural log	$\ln$	Log base e , the default base
Column vector	x in $\mathbb{R}^n$	n numbers stacked vertically

Self-test:

Self-test. Q: The two points give 65 thousand dollars for 150 sq ft. What is the price per sq ft, and what does the round trip check? A: 0.4333 thousand dollars per sq ft. Multiply back: $0.4333 * 150 = 65.0$. The round trip closes, so the unit arithmetic is consistent.

Self-test. Q: A lookup table with arbitrary outputs can fit any input-output pair. Why is that a warning about "a function" as a model? A: A function only promises one output per input. It promises nothing about smoothness, extrapolation, or meaning. The counterexample in U01-C01: any lookup table qualifies, yet it learns nothing.

U02, Linear geometry and decompositions

Data lives in $\mathbb{R}^n$. A matrix is a linear map: it sends the basis vectors somewhere, and the whole space follows. Rank counts the columns that survive. The nullspace holds what the map kills. Eigenvectors keep their direction. SVD splits any matrix into rotation, stretch, rotation: $A = U S V^T$. Covariance is $X^T X$ over n (centered).

Worked number (U02-C03): $v \cdot w = 3.0$, $\|v\| = 5.0$. And (U02-C01): $\text{span}([1, 0], [0, 1]) = \mathbb{R}^2$, dimension 2.

Memory aid. Rank is what survives. The nullspace is what the map kills. SVD always exists.

Object	Shape	One-line meaning
$A: \mathbb{R}^n \rightarrow \mathbb{R}^m$	$m \times n$	Linear map, columns are images of basis vectors
Rank	integer	Number of linearly independent columns
Nullspace	subspace of $\mathbb{R}^n$	All x with $Ax = 0$
Eigenvector v	n -vector	$Av = \lambda v$: direction preserved
SVD: $A = U S V^T$	$U: m \times r, S: r \times r, V: n \times r$	Rotation, stretch, rotation
Covariance	$d \times d$, symmetric, PSD	Spread and correlation of centered features
Condition number κ	scalar	Sensitivity of $Ax = b$ to input error

Self-test:

Self-test. Q: M maps $[1, 1]$ to $[3, 6] = 3 [1, 2]$. Is $[1, 2]$ an eigenvector of M ? A: Yes if $M [1, 2]$ is a scalar multiple of $[1, 2]$. The lesson gives $M @ [1, 1] = 3 [1, 2]$: $[1, 1]$ maps onto the direction $[1, 2]$. Only a full check of M on $[1, 2]$ settles the eigenvector claim. (U02-C08)

Self-test. Q: Why does a threshold-based rank test misfire on noisy data? A: Noise makes every singular value positive. A hard threshold then invents rank. The lesson rule: judge rank from the singular-value gap, not from a cutoff. (U02-C02)

U03, Calculus and optimization

The gradient points uphill, descent steps go the other way. The chain rule carries derivatives through compositions: $dy/dx = dy/dz * dz/dx$. The Hessian classifies curvature. Convex means the Hessian is PSD everywhere, and then every local minimum is global. Taylor turns a hard function into a local quadratic. Gradient descent needs a step size: too large diverges, too small crawls.

Worked number (U03-C01): $f(x, y) = x^2 + 3xy + y^2$ at $(2, 1)$. Analytic: $df/dx = 2x + 3y = 7$, $df/dy = 3x + 2y = 8$. Finite difference with $h = 1e-7$: 6.99999999298484 and 7.999999995789153. Agreement to 8 digits.

Memory aid. Check the gradient against finite differences before you trust any optimizer.

Method	Cost per step	Use it when
Gradient descent	$O(nd)$	Large data, smooth loss, first-order budget
SGD	$O(d)$ per sample	Streaming data, noise tolerance helps
Newton	$O(d^3)$	Small d , exact curvature, quadratic finish
Projected GD	$O(nd)$ + projection	Constraints you can project onto cheaply
Lagrange / KKT	problem-specific	Equality and inequality constraints

Self-test:

Self-test. Q: Why is the agreement between analytic and finite-difference gradients a gate, not a nicety? A: A wrong gradient poisons every optimizer built on top of it. The 8-digit match in U03-C01 proves the paper derivative and the code derivative are the same object. (U03-C12)

Self-test. Q: Convex plus differentiable: where can gradient descent end? A: At the global minimum. Convexity makes every stationary point global. Non-convex problems lose that guarantee. (U03-C05)

U04, Probability, density, and estimation

Uncertainty is a sample space with a measure. PMF for discrete, PDF for continuous, CDF for both. Expectation is the average under the measure variance is the average squared deviation. Joint, marginal, and conditional describe how variables hang together. Entropy measures surprise, KL measures the price of using the wrong distribution minimizing KL is the same as maximizing likelihood. MLE picks the parameters that make the observed data most likely.

Worked number (U04a-SB04): fair die, $E = 3.5$, $\text{Var} = 35/12 = 2.9167$. (U04a-SB07): 8 IID Bernoulli(0.3) draws, seed 7, sample mean 0.5, error 0.2. The estimate $\hat{p} = 0.75$ comes from maximizing the log-likelihood.

Memory aid. Entropy is surprise. KL is the price of the wrong code. MLE spends neither: it picks the parameters the data likes best.

Object	Formula	Role in ML
Expectation	$E[X] = \sum x p(x)$	Population average
Variance	$E[(X - E[X])^2]$	Spread around the mean
Entropy	$H = -\sum p \log p$	Bits to encode the truth
KL divergence	$KL(p \parallel q) = \sum p \log(p/q)$	Cost of coding p with q
MLE	$\text{argmax}_{\theta} \prod p(x_i, \theta)$	Fit by maximizing likelihood
Gaussian MLE	$\hat{\mu} = \text{mean}, \hat{\sigma}^2 = \text{variance}$	Closed form from data

Self-test:

Self-test. Q: Why does minimizing $KL(p_{\text{data}} \parallel p_{\theta})$ equal maximizing the likelihood? A: KL splits into the data entropy (constant in θ) minus the expected log-likelihood. Only the second term moves with θ . So minimizing KL is the same as maximizing likelihood. (U04b-SB11)

Self-test. Q: The 8 draws give mean 0.5 while the true p is 0.3. Is the estimator broken? A: No. The error 0.2 is noise from the small sample at $n = 8$. The lesson computes it exactly: small samples wobble. That is why the block studies the estimator's distribution, not just its value. (U04a-SB08)

U05, Risk, regularization, and generalization

Train on the empirical risk R_{hat} , care about the population risk R . The gap is the generalization problem. Bias is error from a too-simple model, variance is error from a too-sensitive one. Penalties (L1, L2) shrink weights, priors say the same thing in Bayesian words. Capacity must match data: cross-validation measures the match, learning curves plot it, leakage destroys it.

Worked number (U05-C01): 0-1 loss, $R_{\text{hat}}(h) = 0.25$ on the toy. (U05-C04): $p = 0.7$, $n = 4$, IID MSE = 0.0525.

Memory aid. R_{hat} is what you see. R is what you get. Regularization is the bridge toll.

Regularizer	Penalty	Bayesian story
L2 (ridge)	$\lambda \ w\ ^2$	Gaussian prior on w
L1 (lasso)	$\lambda \ w\ _1$	Laplace prior, induces sparsity
Early stopping	stop before convergence	Implicit capacity control
Dropout	random masks	Approximate model averaging

Self-test:

Self-test. Q: High bias or high variance: training error high, and more data does not help? A: High bias. The model class cannot reach the truth. The U05 rule: variance shrinks with data, bias does not. Fix the model, not the sample size. (U05-C04, C10)

Self-test. Q: Why does the lesson treat L2 as a Gaussian prior instead of a hack? A: The MAP estimate with a Gaussian prior is exactly the penalized least-squares objective. Same optimum, two stories. The prior story predicts what the penalty does to each weight. (U05-C06)

U06, Linear models, kernels, and margins

Least squares has a closed form: $w = (X^T X)^{-1} X^T y$. With Gaussian noise it is also the MLE and the MAP with a Gaussian prior: three stories, one optimum. Logistic regression models log-odds as linear softmax extends it to many classes. Kernels replace dot products with $k(x, x')$: linear models in a rich feature space without ever building the features. SVMs maximize the margin, the dual solves it with Gram matrix entries only.

Worked number (U06-C03): $\text{logit}(0.7) = \log(0.7/0.3) = 0.847298$. $\text{sigmoid}(0.847298) = 0.7$. The round trip is exact to float precision. (U06-C02): $\sigma^2_{\text{hat}} = \text{RSS} / n = 1.8 / 4 = 0.45$. Max log-likelihood -4.0787. Predictive variance at $x = 5$: $\sigma^2 (1 + x^T (X^T X)^{-1} x) = 1.665$. The 1 is the new noise, the second term is parameter doubt that grows with distance from the data.

Memory aid. Squared loss is Gaussian noise in disguise. The kernel trick is a dot product in a space you never visit.

Model	Objective	Closed form or algorithm
OLS	$\min \ y - Xw\ _2^2$	$w = (X^T X)^{-1} X^T y$
Ridge	$\min \ y - Xw\ _2^2 + \lambda \ w\ _2^2$	
Logistic	max log-likelihood, sigmoid link	IRLS / gradient methods
SVM (soft)	$\min (1/2) \ w\ _2^2 + C \sum \xi_i$	Dual QP over Gram entries

Self-test:

Self-test. Q: $\phi(2) \cdot \phi(3) = 43$, not 49. What does that number refute? A: The hope that the kernel equals the naive feature dot product. The computed 43 versus the naive 49 shows the kernel defines its own inner product. Trust the Gram matrix, not the intuition. (U06-C06)

Self-test. Q: Remove the Gaussian noise assumption. What happens to squared loss? A: It loses its license. The lesson states it plainly: Laplace noise buys absolute loss instead. The loss follows the noise story, not the other way around. (U06-C02)

U07, Trees and ensembles

A tree partitions space into axis-aligned regions and predicts the majority or the mean in each. Impurity (Gini, entropy) scores a split the best split maximizes the impurity drop. Deep trees memorize: pruning trades depth for honesty. Bagging averages many noisy trees and kills variance. Boosting fits residuals in sequence and kills bias. Feature importance from trees is fragile, baselines come first.

Worked number (U07-C02): node [4, 1], $p = [0.8, 0.2]$. $\text{Gini} = 1 - (0.64 + 0.04) = 0.32$. $\text{Entropy} = 0.6730$.

Memory aid. One tree memorizes. Many trees vote. Boosting corrects mistakes in sequence.

Method	Kills	Price
Single tree	nothing (memorizes)	overfit
Pruning	overfit	bias rises
Bagging / random forest	variance	compute, correlated trees
AdaBoost / gradient boosting	bias	overfit risk, tuning

Self-test:

Self-test. Q: Root Gini is 0.5. The best split gives gain 0.5 with both children pure. Why is that suspicious, not perfect? A: Pure children on training data scream memorization. The lesson computes it and flags it: check the held-out score before celebrating. (U07-C03, C04)

Self-test. Q: Bagging averages B trees. When does the variance drop fail? A: When the trees are correlated. The variance of the average keeps the covariance term. Random forests decorrelate via feature subsampling for exactly this reason. (U07-C06, C07)

U08, Neural and sequence architectures

An MLP is affine maps plus nonlinearities, backprop is the chain rule with a budget. Convolutions share weights across space: locality plus translation structure. Recurrence shares weights across time, and the repeated multiplication explodes or decays gradients: $w = 1.5$ gives $1.5^{10} = 57.665$. Gates (LSTM/GRU) tame it. Attention is a soft lookup: $\text{softmax}(Q K^T / \sqrt{d_k}) V$. The transformer stacks it. Normalization and Adam stabilize the optimization.

Worked number (U08-C07): recurrent weight $w = 1.5$ over 10 steps gives $1.5^{10} = 57.665$. The product explodes. Below 1 it decays to zero. The gradient is a product, so its fate is exponential.

Memory aid. Backprop is the chain rule on a budget. Attention is a soft lookup. The gradient through time is a product: it explodes or it dies.

Architecture	Weight sharing	Handles
MLP	none	fixed-size vectors
CNN	across space	images, local patterns
RNN	across time	sequences, short memory
LSTM/GRU	across time + gates	sequences, long memory
Transformer	none (attention instead)	sequences, sets, long range

Self-test:

Self-test. Q: Why does the lesson compute $1.5^{10} = 57.665$ instead of just saying "gradients explode"? A: The number proves the mechanism. A repeated product with base above 1 grows exponentially in the step count. No story about architecture changes that arithmetic. (U08-C07)

Self-test. Q: Scores row $[0.5, 0, 0.5]$ become weights $[0.3837, 0.2327, 0.3837]$. What did softmax do, and what is the output row? A: Softmax turned the scores into a distribution that sums to 1. The output row is the weighted sum of the value rows: $[3.0, 4.0]$. That is the soft lookup. (U08-C09)

U09, Unsupervised and latent models

No labels: find structure. K-means alternates assignment and centroid update, it needs scaled features because distance decides everything. PCA keeps the top eigenvectors of the covariance: the directions of maximal variance. Reconstruction error measures what you threw away. Evaluate on held-out data: train inertia always flatters. The EM and GMM leaves live in the U04c source block.

Worked number (U09-C03): eigenvalues 6.541656 and below. (U09-C04): PCA with $k = 1$ gives reconstruction MSE 0.009172.

Memory aid. Scale before you cluster. The eigenvalues tell you what PCA kept. Held-out data tells you the truth.

Method	Objective	Trap
K-means	min within-cluster sum of squares	distance needs scaled features
PCA	max projected variance, k directions	variance is not always meaning
GMM + EM	max likelihood of mixture	local optima, degeneracy, init matters
Parzen window	kernel density estimate	bandwidth decides everything

Self-test:

Self-test. Q: Raw features: nearest of A is B. After scaling, the answer changes. Which one is right? A: Neither by default. Distance is unit-dependent, so the clustering follows the units. The lesson rule: scale first, then interpret. The computed example shows the flip. (U09-C02)

Self-test. Q: Train inertia keeps dropping as k grows. Why is that not progress? A: More centroids always fit the training points better. The held-out check in U09-C12 exists because train inertia flatters every k . Model selection needs data the fit never saw.

U10, Generative bridge and mathematical synthesis

A generative model writes the joint distribution $p(x, z)$ and samples from it. The VAE maximizes the ELBO: $E_q[\log p(x|z)] - KL(q || p)$. The GAN plays minimax: a generator fools a discriminator. Samples are not densities: a GAN gives draws without likelihoods, a VAE gives a lower bound on them. Every proof rests on assumptions, the lesson lists them, then breaks them with toy counterexamples.

Worked number (U10-C02): on the discrete toy, one ELBO term is $-0.5 (0.7 - 0.55)^2 - 0.5 \log(2 \pi) = -0.9302$. The ELBO is a sum of such closed-form pieces plus the KL gap.

Memory aid. The VAE bounds the likelihood. The GAN skips it. Samples are not densities.

Model	Objective	Gives you
VAE	$\max \text{ELBO} = E_q[\log p(x z)] - KL(q p)$	samples + likelihood lower bound
GAN	minimax game	samples, no likelihood
GMM + EM (from U04c)	max mixture likelihood	densities, soft assignments

Self-test:

Self-test. Q: The ELBO has two terms. What does each term want? A: $E_q[\log p(x|z)]$ wants reconstructions that explain the data. $KL(q || p)$ wants the encoder to stay near the prior. The tension between them is the whole model. (U10-C02)

Self-test. Q: A paper reports great samples from a GAN. Can you compare its likelihood to a VAE? A: No. The GAN gives no likelihood, the VAE gives a lower bound. The lesson rule: samples versus densities is a category distinction, not a quality ranking. Compare like with like. (U10-C05)

Where to go next

- Full lessons, labs, and interview banks: the course home.
- One dense page: the cheatsheet.
- Read the shared prerequisite bridges P01-P24 before the unit that needs them. The course home maps each unit to its modules.

Cheatsheet: formulas, shapes, traps, decision rules

One dense page for the whole course. Formulas use the notation fixed in notation and shapes. Section pointers name the lesson that proves each line.

U01, mathematical language

Formula / shape	Trap	Decision rule
$\sum_i x_i$: add over i	A bare sum with no index range hides the set	Write the index set on first use
$\ln$: log base e	log base 10 in code, base e on paper	State the base once, then hold it
x in $\mathbb{R}^n$: column vector	Row-vector code breaks matrix products	Keep vectors as columns in formulas
price = 65 / 150 = 0.4333 per sq ft	Unit mismatch silently scales answers	Check units before you check code
Round trip: 0.4333 * 150 = 65.0	One-way arithmetic hides factor errors	Multiply back, the trip must close

U02, linear geometry

Formula / shape	Trap	Decision rule
$A: m \times n$ maps $\mathbb{R}^n \rightarrow \mathbb{R}^m$	Transpose confusion flips domain and codomain	Read columns: they are images of basis vectors
$\text{rank}(A)$: independent columns	Noise makes every singular value positive	Judge rank from the singular-value gap
Nullspace: $\{x: Ax = 0\}$	Forgetting it loses solutions to $Ax = b$	Check the nullspace before you invert
$Av = \lambda v$	Not every matrix has a full eigenbasis	Symmetric matrices do, use them when you can
$A = U S V^T$, $U: m \times r$, $S: r \times r$, $V: n \times r$	Truncation rank picked by habit	Keep the singular values that carry the energy
Covariance: $d \times d$, symmetric, PSD	Uncentered data corrupts it	Center first, then form $X^T X / n$
$\kappa(A)$: sensitivity of $Ax = b$	Large κ amplifies input error	Rescale or regularize before you solve
Projection: $(b \cdot a) / (a \cdot a) \cdot a$	Forgetting to divide by $(a \cdot a)$	Normalize the direction, then project

U03, calculus and optimization

Formula / shape	Trap
df/dx via chain rule: $dy/dx = dy/dz * dz/dx$	Dropped link in a long chain
Gradient: vector of partials	Gradient is not the direction of steepest ascent in a warped m
Hessian PSD everywhere $\Rightarrow$ convex	One non-PSD point kills global guarantees
Taylor: $f(x) \approx f(a) + \text{grad } f(a)^T (x-a) + 1/2 (x-a)^T H (x-a)$	Trusting the quadratic far from a
GD: $w \leftarrow w - \eta \text{ grad}$	η too large diverges, too small crawls
Newton: $w \leftarrow w - H^{-1} \text{ grad}$	$O(d^3)$ per step, indefinite H misleads
KKT: stationarity, primal and dual feasibility, complementary slackness	Ignoring complementarity
Gradient check: analytic vs finite difference to 8 digits	Skipping it

U04, probability and estimation

Formula / shape	Trap	Decision rule
$E[X] = \sum x p(x)$, $\text{Var} = E[(X - E[X])^2]$	Confusing sample and population moments	Name which distr
$p(x, y) = p(x$	$y) p(y)$	Conditioning on th
$H = -\sum p \log p$	Entropy is not uncertainty about the world	It is the code leng
$KL(p$		$q) = \sum p \log(p/$
$\min KL(p_{\text{data}}$		$p_{\text{theta}}) = \max \text{lik}$
MLE: $\text{argmax}_{\text{theta}} \prod_i p(x_i, \text{theta})$	Maximizing density at a point for continuous data	Work with the log
Gaussian MLE: $\mu_{\text{hat}} = \text{mean}$, $\sigma^2_{\text{hat}} = \text{variance of data}$	Biased variance at small n	Know the n vs $n-1$
Bernoulli MLE: $p_{\text{hat}} = k / n$	$n = 8$ gives wobble (0.5 vs 0.3 in the lesson toy)	Report the estima
EM: E-step fills latent posteriors, M-step maximizes	Local optima	Restart from seve

U05, risk and generalization

Formula / shape	Trap	Decision rule
$R_{\text{hat}}(h) = (1/n) \sum \text{loss}(h(x_i), y_i)$	Treating it as the goal	It is the proxy, R is the goal
$R(h) = E[\text{loss}(h(x), y)]$	You never observe it	Bound it with validation
Bias: error from a too-simple class	Adding data to fix bias	Bias does not shrink with n, change the model
Variance: error from a too-sensitive fit	Adding capacity to fix variance	Variance shrinks with n, or regularize
L2: + lambda		w^2 lar
L1: + lambda		w _1 Sp
k-fold CV	Leakage across folds	Split before any preprocessing that sees labels
Learning curves	Reading noise as signal	Plot mean and spread across seeds
Leakage	Test info in training features	Audit the feature pipeline, not just the score

U06, linear models, kernels, margins

Formula / shape	Trap
OLS: $w = (X^T X)^{-1} X^T y$	$X^T X$ singular
Squared loss = Gaussian noise MLE	Assuming the noise story
$\text{logit}(p) = \log(p/(1-p))$, $\text{logit}(0.7) = 0.847298$	Forgetting the link
Softmax: $\exp(z_i) / \sum_j \exp(z_j)$	Overflow in exp
Kernel: $k(x, x')$ replaces dot products	Assuming the kernel equals a naive feature dot product (43 vs 49 i
Gram K: $n \times n$, symmetric, PSD	Non-PSD K breaks the dual
SVM dual: solve in α , decide with $\sum \alpha_i y_i k(x_i, x) + b$	Tuning C on training accuracy
$\kappa(X^T X)$ raw 3.350789e6 in the lesson	Solving the normal equations directly

U07, trees and ensembles

Formula / shape	Trap	Decision rule
Gini = $1 - \sum p_k^2$, node [4,1]: 0.32	Gini 0 with impure children elsewhere	Score the split, not the node
Entropy = $-\sum p_k \log p_k$, node [4,1]: 0.6730	Log base changes the number	Fix the base before you compare
Split gain = impurity(parent) - weighted impurity(children)	Best split on training = best model	Validate the depth, not the gain
Pruning: trade depth for honesty	Pruning by gut	Prune on validation error
Bagging: average B bootstrapped trees	Correlated trees keep the covariance term	Subsample features to decorrelate
Boosting: fit residuals in sequence	Chasing noise	Shrink the step, stop on validation
Imbalance: class weights or resampling	Accuracy on skewed classes	Report per-class scores and metrics
Feature importance from trees	Fragile under correlated features	Treat it as a hint, not a ranking

U08, neural and sequence

Formula / shape	Trap	Decision rule
MLP: affine, nonlinearity, repeat	Dead ReLUs	Watch the activation histogram
Backprop: chain rule with a budget	Wrong gradient, wrong everything	Finite-difference check first (U03-C12)
Conv: shared kernel across space	Padding changes the shape	Track shapes through every layer
RNN: same weights each step	Gradient product explodes or dies ($1.5^{10} = 57.665$)	Gate it or shorten the horizon
LSTM/GRU gates	Gate saturation	Initialize to let signal through early
Attention: $\text{softmax}(Q K^T / \sqrt{d_k}) V$	Forgetting the $\sqrt{d_k}$ scale	Scores grow with d_k , scale them
Attention weights sum to 1 per row	Reading weights as explanations	Weights show routing, not reasons
Transformer: stacked attention + MLP	Quadratic cost in sequence length	Budget n^2 before you scale n
Normalization (batch/layer)	Train/test statistic mismatch	Know which statistics each mode uses
Adam vs SGD	Adam's adaptivity hides bad conditioning	Compare on wall-clock, not steps

U09, unsupervised and latent

Formula / shape	Trap	Decision rule
K-means: alternate assign and update	Distance decides, units decide distance	Scale features first
PCA: top k eigenvectors of covariance	Variance is not meaning	Inspect what the component
Reconstruction MSE at k = 1: 0.009172 (lesson toy)	Judging by train error only	Evaluate on held-out data
Eigenvalues: 6.541656 and below (lesson toy)	Keeping components by habit	Cut where the spectrum drop
GMM + EM	Degeneracy: a component collapses on one point	Constrain or regularize the c
Held-out inertia	Train inertia always flatters	Select k on data the fit never

U10, generative synthesis

Formula / shape	Trap	Decision rule
ELBO = $E_q[\log p(x z)] - KL(q p)$	Posterior collapse	Check that the latent codes carry information
VAE: encoder q , decoder p , prior $p(z)$	Mode collapse	Watch sample diversity, not just quality
GAN: min_G max_D game	Comparing GAN samples to VAE likelihoods	Compare like with like
Samples are not densities	Citing the theorem past its assumptions	List the assumptions, then test them
Every proof rests on assumptions	Arguing from intuition	Build the smallest case that breaks the claim
Toy counterexample beats a vague objection		

Cross-unit decision rules

Situation	Rule
Loss will not decrease	Check the gradient against finite differences (U03-C12), then the learning rate, then the data pipeline
Model choice under time pressure	Fit OLS/logistic first, kernels when n is small and features are weak, trees when features are mixed
A number looks too good	Check for leakage (U05-C11), then the split, then the metric
A proof is quoted	Ask for the assumptions (U10-C07), test them on a toy (U10-C08)
A figure is shown	Ask for the code and the axes (U01-C09), honest plots carry both